Java 25 Features for Patterns

Model Payment ADT without illegal states
Season 0 · Language · Beginner

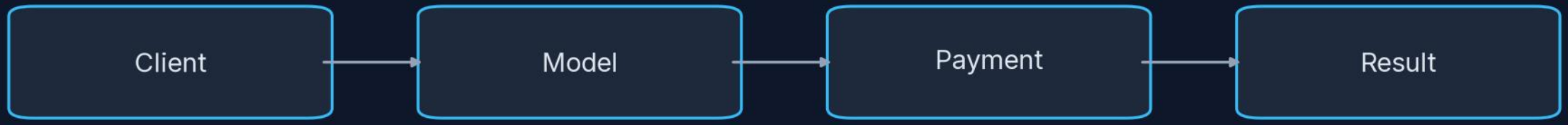
Today's Hook

- Problem: Model Payment ADT without illegal states
- Naive pain cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- We will refactor to Java 25 Features with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: Model Payment ADT without illegal states
- Constraints: solve «Model Payment ADT without illegal states» with Java 25 Features — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Model Payment ADT without illegal states
- Success looks like: card:10

Structure



Java 25 Features: Model Payment ADT without illegal states

Real-World Scenario

- Running example: Model Payment ADT without illegal states
- Key types: Payment, Card, Cash, Wire
- Happy-path stdout cue: card:10
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.functional.adt.problem;

import java.util.*;

public final class NaivePay {
  public static String describe(String type, int cents){
    if("card".equals(type)) return "card:"+cents;
    if("cash".equals(type)) return "cash:"+cents;
    return "other";
  }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Model Payment ADT without illegal states
- Without Java 25 Features, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.functional.adt.problem;

import java.util.*;
import java.util.function.*;

sealed interface Payment permits Card, Cash, Wire {}
record Card(String last4, int cents) implements Payment {}
record Cash(int cents) implements Payment {}
record Wire(String iban, int cents) implements Payment {}
final class Payments {
  static String describe(Payment p){
    return switch (p) {
      case Card(String last4, int cents) -> "card ****"+last4+" "+cents;
      case Cash(int cents) -> "cash "+cents;
      case Wire(String iban, int cents) -> "wire "+iban+" "+cents;
    };
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.adt.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== ADTs: exhaustive Payment describe ===");
    System.out.println("--- naive ---"); System.out.println(NaivePay.describe("card", 10));
System.out.println("--- ADT ---");
System.out.println(Payments.describe(new Card("1234", 1999)));
System.out.println(Payments.describe(new Cash(500)));
System.out.println(Payments.describe(new Wire("DE89", 10000)));
  }
}
```

Walkthrough Output

```
=== ADTs: exhaustive Payment describe ===
--- naive ---
card:10
--- ADT ---
card ****1234 1999
cash 500
wire DE89 10000
```

Step-by-Step Trace

- Match stdout to stages — look for: card:10
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 01-java25-features ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.adt.problem;

import java.util.*;
import java.util.function.*;

sealed interface Payment permits Card, Cash, Wire {}
record Card(String last4, int cents) implements Payment {}
record Cash(int cents) implements Payment {}
record Wire(String iban, int cents) implements Payment {}
final class Payments {
  static String describe(Payment p){
    return switch (p) {
      case Card(String last4, int cents) -> "card ****"+last4+" "+cents;
      case Cash(int cents) -> "cash "+cents;
      case Wire(String iban, int cents) -> "wire "+iban+" "+cents;
    };
  }
}
```

Trade-offs

- Use Java 25 Features when «Model Payment ADT without illegal states» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Java 25 Features in product language.

Related Patterns

- Name the pattern usually confused with Java 25 Features and separate the forces.
- Sketch the roles for «Model Payment ADT without illegal states» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Model Payment ADT without illegal states
- Naive pain → Java 25 Features boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.